

19 — Contare le righe non è conoscere la scala

La domanda a cui risponde: hai una struttura ad albero disegnata a righe, e la posizione della riga *sembra* dire a che livello sei. Quando smette di dirlo, come te ne accorgi — e cosa metti al suo posto?

Per contorno: come si distingue un dato assente da un dato che vuol dire “no”, e perché in un dataset multilingua l’ordine alfabetico dei file è una scelta di lingua travestita da dettaglio tecnico.

Il fatto

Una segnalazione di tre parole: «*Linea di Omastar è sbagliata, c’è un fase1, fase2, manca il base e c’è un turbo*».

La finestra della linea evolutiva (`src/ui/linea-evolutiva/`) mostrava questo:

riga	etichetta scritta	carta	stadio vero sulla carta
0	Base	Omanyte	Livello 1
1	Livello 1	Omastar	Livello 2
2	Livello 2	Omastar TURBO	TURBO , che non è un livello

Tre difetti diversi che si erano sommati in un unico schermo storto. È un caso di studio utile proprio per quello: un bug che sembra uno solo, e invece sono tre cause indipendenti che convergono sulla stessa carta.

Parte 1 — L’indice implicito

Il codice faceva questo:

```
const ETICHETTE = ['Base', 'Livello 1', 'Livello 2'];
const etichetta = ETICHETTE[gradino.livello];
```

dove `gradino.livello` era **la posizione nell’array dei gradini**. Funziona finché vale un’assunzione mai scritta da nessuna parte:

la riga più bassa è sempre un Base, e ogni riga sale di esattamente uno.

Questa assunzione è **quasi** sempre vera, ed è il tipo peggiore. Nel dataset ci sono almeno due famiglie che la smentiscono:

- **Omanyte** è stampato `Livello 1`. Il gradino sotto di lui esiste ma è un *fossile*, che è una carta **Allentore**: la linea comincia legittimamente dal primo piano.
- **Pichu** e **Pikachu** sono **due carte Base**. Il “baby Pokémon” non è uno stadio in meno: è un Base da cui un altro Base evolve.

Il rimedio non è aggiustare l’aritmetica, è **smettere di dedurre un dato che si possiede già**. In `src/engine/catena.js` ogni gradino porta adesso due numeri distinti:

```
/**
 * @property {number} livello la riga: 0 la più bassa mostrata, poi 1 e 2
 * @property {number} stadio lo stadio di gioco (0 Base, 1, 2)
 */
```

`livello` serve a disegnare (è la chiave del registro delle carte a schermo), `stadio` serve a **dire** (è quello che va nell’etichetta). Che per la maggior parte delle linee coincidano è una coincidenza, non un’identità.

Nota per chi viene da Java. È la differenza fra l’indice di una `List` e il valore di una `enum`: `ordinal()` sembra utile finché qualcuno non riordina le costanti. La regola è la stessa — non usare la posizione come se fosse un significato.

Il difetto che si nascondeva dietro

La stessa assunzione era usata anche per **filtrare** i figli, non solo per etichettarli:

```
const atteso = livelloDelPrimo + gradini.length; // prima
```

Con Pichu al piano terra, la terza riga pretendeva uno stadio 2, e **Raichu — che è un Livello 1 — veniva scartato**. La linea di Pikachu perdeva Raichu e al suo posto pescava Raichu *ex*, che nel dataset risulta Livello 2. Nessuno se n'era accorto: la finestra mostrava *una* carta plausibile al posto giusto.

Il rimedio è legare l'atteso al dato, non al conteggio:

```
const atteso = gradini[gradini.length - 1].stadio + 1; // adesso
```

Morale generale: quando una deduzione sbagliata **etichetta** male, si vede subito; quando la stessa deduzione **filtra**, sparisce roba in silenzio. Cerca sempre il secondo uso di un'assunzione che hai appena trovato sbagliata.

Parte 2 — Assente non vuol dire “no”

Il TURBO. `src/engine/stadi.js` sapeva già che TURBO non è uno stadio canonico:

```
const ESOTICI = { VMAX: 1, 'V ASTRO': 1, MEGA: 1, TURBO: 1, 'V UNIONE': 0, Ricreato: 0 };
```

E `data/evoluzioni.json` conteneva già un indice `stadi` con lo stadio numerico di ogni specie. La tentazione era ovvia:

```
// SBAGLIATO
if (!(nome in stadi)) continua; // niente stadio → sarà un TURBO
```

Ma `omastar turbo` manca da `stadi` insieme ad altre **229 specie**, quasi tutte per un motivo completamente diverso: nessuna loro stampa dichiara lo stadio. L'assenza di un dato non è un valore: è l'unione di tutti i motivi per cui quel dato può mancare.

Serviva quindi renderlo **esplicito**, esattamente come il progetto aveva già fatto per i fossili con `nonPokemon`:

```
const NON_GRADINI = new Set(['TURBO', 'VMAX', 'V ASTRO', 'MEGA', 'V UNIONE']);
// ...
const indice = { da, nonPokemon, stadi, esotici };
```

Nota chi resta **fuori** dall'elenco: **Ricreato**. È il fossile rianimato, si mette in gioco dalla sua carta Allenatore e **occupa davvero un gradino**. Un elenco di esclusione va compilato guardando cosa fa ogni voce nel gioco, non guardando quali nomi “suonano speciali”.

Il campo nuovo su un file già in cache

`data/evoluzioni.json` sta nel guscio del service worker, quindi su un telefono già installato può esistere una copia **vecchia**, senza esotici. Il modulo che lo legge lo mette in conto:

```
cacheEsotici = new Set((nuovo ? indice.esotici ?? [] : []).map(normalizza));
```

Un Set vuoto degrada al comportamento di prima — sbagliato ma non rotto — invece di far esplodere `undefined.has()`. È lo stesso trattamento che `stadi` aveva ricevuto quando era stato aggiunto a sua volta: in una PWA **ogni campo nuovo di un file cacheato è un campo opzionale**, per almeno una versione.

Parte 3 — Il Base che non manca: si chiama Allenatore

La terza causa. `catenaVersoIlBasso` si fermava correttamente sul fossile:

```
if (nonPokemon.has(chiave)) break; // e il nome finiva nel nulla
```

Fermarsi era giusto — stampare *Vecchio Helixfossile* come Pokémon Base è un bug che nel progetto era già successo — ma **buttare via il nome** trasformava un “qui c'è una carta di un'altra categoria” in un “qui non c'è niente”. All'utente arrivava una linea che comincia a metà, cioè una linea rotta.

La correzione è di una riga e mezza:

```
if (nonPokemon.has(chiave)) {  
  origine = precedente;    // non è un gradino, ma esiste ed è la risposta  
  break;  
}
```

e la finestra ora lo dice: «Questa linea non ha un Pokémon Base: si mette in gioco da Vecchio Helixfossile, che è una carta Allenatore.»

C'è un principio generale sotto, e vale ben oltre questo caso: **un break che scarta l'informazione per cui si è interrotto il ciclo è quasi sempre un bug in attesa**. Ti sei fermato *per un motivo*; quel motivo è un dato.

La firma che cambia senza rompere i chiamanti

catenaVersoIlBasso era usata anche da src/engine/linee.js, che dei fossili non sa che farsene. Invece di cambiare il tipo di ritorno per tutti:

```
function scendi(carta, indice, nonPokemon) { /* ... */ return { catena, origine }; }  
  
export function catenaVersoIlBasso(carta, indice, nonPokemon) {  
  return scendi(carta, indice, nonPokemon).catena;  
}
```

La funzione ricca è privata, quella vecchia resta identica ed è un guscio di una riga. Chi non ha bisogno del dato nuovo non deve nemmeno sapere che esiste.

Parte 4 — L'ordine alfabetico come scelta di lingua

Con la nota a schermo è saltato fuori un difetto che prima nessuno poteva vedere, perché quel nome non veniva mostrato da nessuna parte:

«si mette in gioco da **Helix Fossil**»

su una carta italiana. Il progetto ha una regola precisa su questo (docs/apprendimento/14-una-lingua-che-manca.md): il danno non è l'inglese, è l'inglese spacciato per italiano.

La causa era in tools/genera-indice-evoluzioni.mjs. Lo stesso fossile ha nomi diversi in set diversi, e in caso di conflitto l'indice teneva **il primo che incontrava**:

```
const file = readdirSync(CARTELLA_SET).sort();    // "dp5" viene prima di "sv03.5"
```

Il criterio “il primo” non era una decisione: era l'ordine alfabetico dei nomi dei file, che per puro caso mette i set inglesi vecchi prima di quelli italiani recenti. Adesso il criterio è dichiarato:

```
if (daSetInglese.get(chiave) && !inglese) {  
  evoluzioni.set(chiave, carta.evolveDa);    // l'italiano scalza l'inglese  
}
```

Dieci valori sono cambiati, e sono esattamente i dieci fossili. Il conflitto continua a essere **segnalato** nel log: preferire non è nascondere.

Da portarsi via. Ogni volta che scrivi “si tiene il primo”, chiediti *primo secondo quale ordine, e chi lo ha deciso*. Se la risposta è “il filesystem”, non hai un criterio: hai un caso.

Parte 5 — Il refuso che si travestiva da fossile

Un'ultima trappola, incontrata mentre si allargava l'elenco nonPokemon a tutti i nomi di pre-evoluzione mai dichiarati. La regola era:

```
// un nome che non è una specie Pokémon è una carta Allenatore
```

Sbagliata. Fra quei nomi ci sono anche i **refusi** del dataset: Drowsee per Drowzee, Tailow per Tailow, Jigglypuff. Con quella regola, Hypno avrebbe ricevuto la nota «*si mette in gioco da Drowsee, che è una carta Allenatore*» — una frase falsa, generata con la stessa sicurezza di quella vera.

La correzione è chiedere una **prova positiva** invece di una negativa: il nome deve corrispondere a una carta non-Pokémon davvero stampata nel dataset.

```
!speciePokemon.has(normalizza(nome)) &&  
  (carteAllenatore.has(normalizza(nome)) || vincitori.has(nome))
```

!A e B non sono la stessa cosa, anche quando su tutti gli esempi che hai in mente coincidono. Il secondo ramo (vincitori) conserva i pochi casi storici senza carta corrispondente — Nidoran?, Rocket's Meowth — che l'elenco vecchio già copriva: una correzione non deve perdere quello che funzionava.

I file toccati

file	cosa fa adesso
tools/genera-indice-evoluzioni.mjs	scrive esotici, tiene tutti i nomi dei fossili, preferisce l'italiano
src/data/dataset.js	espone speciEsotiche(), tollera il file vecchio senza il campo
src/engine/catena.js	stadio per gradino, origine, esotici fuori dai gradini
src/ui/linea-evolutiva/	etichetta dallo stadio, nota sull'origine
tests/evoluzioni-indice.test.js	difende il file generato: un campo che manca non deve tacere

Domande di verifica

1. **Due numeri per una riga.** gradino.livello e gradino.stadio adesso convivono. Trova nel componente i punti in cui si usa l'uno e quelli in cui si usa l'altro, e di' cosa si romperebbe scambiandoli. (Suggerimento: guarda la chiave del #registro.)
2. **L'assenza ambigua.** Elenca tre motivi diversi per cui una specie può mancare da stadi in data/evoluzioni.json. Poi cerca nel progetto un altro campo la cui assenza è ambigua allo stesso modo, e proponi come renderlo esplicito.
3. **Il break che scarta.** Cerca nel progetto altri break o return dentro un ciclo che si interrompono per una condizione senza conservare il valore che l'ha causata. Per ciascuno, di' se quel valore servirebbe a qualcuno.
4. **La compatibilità all'indietro.** Un telefono ha in cache la versione di evoluzioni.json senza esotici ma con il codice nuovo. Descrivi esattamente cosa vede l'utente aprendo la linea di Omastar, e di' se ti sembra una degradazione accettabile.
5. **Il criterio nascosto.** tools/genera-indice-nomi.mjs e tools/completa-ristampe.mjs scelgono anche loro fra più stampe della stessa carta. Vai a vedere con quale criterio: è dichiarato o è l'ordine dei file?
6. **Prova positiva.** Riscrivi il filtro di nonPokemon usando **solo** carteAllenatore, senza il ramo vincitori. Lancia i test e di' quali casi si perdono; poi decidi se li reintrodurresti o se cambieresti i dati.
7. **Il Ricreato.** Ricreato è escluso da NON_GRADINI ma incluso in ESOTICI di stadi.js con livello 0. Spiega perché le due decisioni non si contraddicono, e cosa succederebbe alla linea di Kabuto invertendone una.
8. **La regressione invisibile.** Il caso Pichu → Pikachu → Raichu era rotto *prima* di questa segnalazione, e nessuno l'aveva notato. Scrivi il test che l'avrebbe scoperto senza che nessuno avesse mai aperto quella finestra.